NPCNPGLR

Big Data Testing Framework for Recommendation Systems in e-Science and e-Commerce Domains

Meryem Uzun-Per [a,b,*], Ali Burak Can [a,†], Ahmet Volkan Gürel [a,‡], Mehmet S. Aktaş [c,§]

[a] *BiletBank Research and Development Center, Akdeniz PE-TUR A.Ş., Istanbul, Turkey*
[b] *Computer Engineering Department, Istanbul Health and Technology University, Istanbul, Turkey*
[c] *Computer Engineering Department, Yildiz Technical University, Istanbul, Turkey*
*meryem.uzunper@petour.com, †aliburak.can@petour.com, ‡ahmetvolkan.gurel@petour.com, §aktas@yildiz.edu.tr

***Abstract*—Software testing is an important process to evaluate whether the developed software applications meet the required specifications. There is an emerging need for testing frameworks for big data software projects to ensure the quality of the big data applications and satisfy the user requirements. In this study, we propose a software testing framework that can be utilized in big data projects both in e-science and e-commerce. In particular, we design the proposed framework to test big data-based recommendation applications. To show the usability of the proposed framework, we provide a reference prototype implementation and use the prototype to test a big data recommendation application. We apply the prototype implementation to test both functional and non-functional methods of the recommendation application. The results indicate that the proposed testing framework is usable and efficient for testing the recommendation systems that use big data processing techniques.**

***Index Terms*—testing framework, testing for big data projects, recommendation systems, big data algorithms, distributed systems, Spark MLlib**

I. Introduction

As technology grows and people are involved in using them, the stored data grows daily. Utilizing these data and interpreting meaningful results from them requires an ability to process and test big data applications. Big data, which may be a combination of structured, semi-structured, and unstructured data, is generally characterized by variety, volume, and velocity [1], [2]. Variety refers to the type of data as structured or unstructured. Volume indicates the large size of the data. Velocity refers to data generation and dynamic aspects of data.

Designing and implementing real-time applications on big data is a challenging task [3]. A testing framework for big data applications may reduce the effort to design and execute experimental studies for evaluating the performance of these applications [4]. Most of the existing testing frameworks are not designed for testing the big data applications that utilize Hadoop Distributed File System (HDFS) like systems that store and process data in distributed setting [5]. We argue that there is an emerging need for big data testing frameworks that can evaluate big data applications for varying metrics such as performance from the perspective of latency on a single node, system scalability under increasing data size, system scalability under increasing load, system scalability under increasing users, system speed-up when increasing the number of nodes in datacenters.

One of the major application categories of big data is the recommendation system applications. The primary purpose of recommendation engines is to reduce the information overload that customers face when exposed to large numbers of products or services and introduce them to users with personalized recommendations. Recommendation engines are critical for large-scale applications both in e-commerce [6], [7], [8] and in e-science [9], [10]. To this end, in this study, we focus on designing and implementing a testing framework for recommendation systems that uses big data processing libraries.

The performance of a news recommendation engine is measured by considering speed and precision metrics in [6]. Key problems affecting the performance of recommendation engines include cold start problem, sparsity, diversity, and scalability [11], [12]. Scalability has become the most critical problem among these problems, with the widespread use of applications containing large amounts of data today. The scalability problem directly affects the speed of the recommendation engine and the cost of the resources it needs. There are a few frameworks to compare recommendation engines [13]–[15]. To the best of our knowledge, there is no testing framework designed for testing the recommendation systems that run on large scale data. In particular, existing testing frameworks do not provide comprehensive solutions for evaluating the scalability aspects of recommendation systems.

In this study, we focus on designing and implementing a testing framework that can evaluate both functional requirements and non-functional requirements of the big-data based recommendation systems. We aim to find answers to the following research questions:

- How should a distributed testing framework software architecture be for testing recommendation systems using Hadoop-based big data techniques?
- How should empirical evaluation be done to understand whether the proposed testing framework software architecture is successful in testing the big-data based recommendation systems?
- When testing the recommendation systems using Hadoop-based big data techniques with the help of the

proposed testing framework prototype, Which recommendation algorithms are discovered to be scalable?

In this paper, in order to address the research questions above, we designed and implemented a testing framework for testing recommendation systems using big data processing techniques. **We outline the contributions of this paper as follows:** We propose an abstraction layered software architecture for a big data testing framework that can be used to test both functional and non-functional requirements of the big data-based recommendation systems. We developed a reference prototype application. To illustrate the use of the proposed framework, we also designed and implemented a recommendation system that utilizes big data libraries. With the help of the prototype of the testing framework, we conducted comprehensive testing of the recommendation system and shared the details of the whole process in this study. As part of the recommendation system, we tested four different algorithms as follows: Alternating Least Squares (ALS) [16], Singular Value Decomposition (SVD) [17], User-based Collaborative Filtering (CF) [18], Item-based CF [19].

In the next section, we give background information and literature review in detail. In Section III, we explain our proposed testing framework layered software architecture. In Section IV, we give the details of the prototype implementation and the details of the testing experiments and evaluations. Finally, we conclude our study in Section V.

II. Background and Literature Review

In this section, first, we explain some background information on big data processing platforms and recommendation systems. Then, in the literature survey subsection, we discuss the relevant studies on testing frameworks for big-data based recommendation systems.

A. Background

Open Source Big Data Processing Platforms: Open source, big data processing ecosystem may be categorized into three as data storage, data processing, and workflow management. Open source technologies such as Apache Hadoop, Apache Spark, Apache Flink, and Apache Storm are the major big data processing systems that use The Hadoop Distributed File System (HDFS) as storage and Yet Another Resource Negotiator (YARN) as resource manager and job scheduler. For machine learning implementations, Apache Mahout and Apache Spark are commonly used on Hadoop Distributed File System.

Apache Mahout was a pioneer in scalable machine learning in 2008, which was implemented on top of Apache Hadoop by using the MapReduce paradigm [20]. Although Mahout is mature and supplies many machine learning (ML) algorithms, it is slow due to disk accesses and cannot handle iterative jobs very well. MLlib is a scalable machine learning library on Apache Spark [21]. Spark avoids writing to disk as much as possible and handles iterative operations faster than Hadoop. Since machine learning algorithms generally contain iterative operations, Spark MLlib runs these algorithms faster than Mahout. Both Mahout and Spark provide recommendation algorithms using Spark MLlib libraries.

Recommendation Systems: Recommendation systems aim to predict user preference for an item and provide personalized services [7], [8]. These systems are used both in e-commerce [6], [7], [8] and in e-science [9], [10].

There are many reasons for service providers to use these systems, such as to increase the number of the sold items, sell more diverse items, increase user satisfaction, and understand what the users want [8]. There are two types of feedback that may be gathered to understand users' preferences [7], [18]. In explicit feedback, users directly rate an item positively or negatively, which may lead to more accurate inferences for a user. However, most of the users do not tend to give feedback. In the implicit feedback, other indicators like the number of views, clicks, purchases, etc., are gathered. The implicit feedback may be misleading; for example, an item may be purchased as an order of someone else, a TV show may be open while the viewer is in sleep, and also there is no negative feedback [22]. Still, it is frequently used since it does not require an additional effort of users and is gathered easier.

Content-based filtering, collaborative filtering, and knowledge-based filtering are some types of recommendation systems. The collaborative filtering (CF) approach is mostly used on big data processing platforms due to its parallelization property. Below, we explain the recommendation algorithms which are frequently used on big data.

User-based CF: User-based CF algorithm predicts the rating of a user for an item based on the similarity of users [18]. This approach is also known as nearest neighbor-based CF. There are several methods to calculate the similarity of the users, such as cosine similarity, cosine distance, Pearson correlation, etc. Cosine similarity is a measure of similarity between two vectors. Let x and y denote two vectors, cosine similarity between x and y is represented by

$$cos(\theta) = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}} \tag{1}$$

where x_i and y_i are components of the vectors x and y, respectively. Cosine similarity produces a value between -1 and 1, where $cos(\theta) = 1$ indicates maximum similarity, and $cos(\theta) = 0$ corresponds to orthogonality or lack of correlation, and $cos(\theta) = -1$ denotes maximum dissimilarity. Cosine distance is calculated by subtracting cosine similarity from 1.

Pearson correlation is the cosine similarity between centered versions of two vectors. Let x and y denote two vectors, Pearson correlation coefficient between x and y is represented by

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \overline{y})^2}} \tag{2}$$

where x_i and y_i are components of the vectors, and $\overline{x}$ and $\overline{y}$ indicate the average of the vectors x and y, respectively.

Pearson correlation is considered the most efficient similarity metric since it normalizes the ratings of users. Still, it may differ according to the problem and the application.

Item-based CF: Item-based CF algorithm predicts the rating of a user for an item based on the similarity of items [19]. The exact similarity metrics used in the user-based CF are used for item-based CF. Generally, item-based CF produces better results than user-based CF since user-based CF suffers from sparsity and scalability. Still, both user-based CF and item-based CF may suffer from the cold start problem.

Alternating Least Squares (ALS): ALS algorithm is a matrix factorization algorithm which is used to predict missing values of rating matrices [16], [23]. Let R denote user versus item matrix. By ALS, R is decomposed into two factor matrices as users (U) and items (I). Since both U and I are unknowns, the ALS algorithm starts with random values of U and fixes it to solve I by least squares analysis. Then I is fixed to recompute U matrix. These two steps are repeated until convergence. After convergence multiplication of U and I gives the R matrix with the predicted ratings. The advantages of ALS are parallelization of the algorithm and solving the cold start problem.

Singular Value Decomposition (SVD): SVD is a matrix factorization algorithm which decomposes correlated variables into uncorrelated ones to better show the relationship of the variables [24]–[26]. SVD, which is frequently used in dimensionality reduction problems [27] is also used for recommendation systems, especially in scalable ones [17], [28]. Since SVD scales well for big data, it may be preferred for big data recommendation systems. Let R denotes a rating matrix, SVD decomposes this matrix into three as

$$R = U\Sigma V^T \tag{3}$$

where U and V are unitary matrices and Σ is a diagonal matrix of singular values. In recommendation systems, after decomposing the matrix into three, the low energy singular values of them are eliminated, and the matrices are used to reconstruct the R matrix to predict the missing part of the rating matrix.

B. Literature Survey

Modern recommendation engines working with very fast-growing data are expected to produce the highest volume of output with the lowest possible latency. A recommendation engine that works efficiently with a small number of users and items should be able to continue to work efficiently even when the number of users and items grows [29]. Various approaches have been proposed in the literature to obtain highly scalable recommendation engines.

There were some early scalable recommendation engines before Mahout and MLlib were used. In 2002, a scalable recommendation engine was proposed based on parallelization of SVD that uses dimensionality reduction techniques [28]. In 2007, a recommendation engine based on CF and MapReduce paradigm was proposed for personalized recommendation on Google News, serving several million unique visitors per week [30].

Since 2009, relatively better scalability performance has been produced when CF-based recommendation engines are supported by Apache Hadoop Mahout library [31], [32]. Since 2014, Apache Spark MLlib, which produces higher scalability performance than Apache Mahout, has emerged and started to be used in recommendation engines [6], [33]. A scalable ALS and CF-based approach is proposed for product recommendations in [3], and for movie recommendations in [34].

We observe some studies that address the need for big data testing frameworks [4], [5]. For example, Alexandrov et al. proposed a testing framework to test and validate the generation of big data [4]. In another example, Li et al. proposed a scalable big data testing framework to test extract, transform and load applications on big data [5]. Alexandrov et al. discuss issues in big data testing and benchmarking [4]. In their study, they also propose a testing framework to test and validate the generation of big data. They argue that a testing framework for big data may reduce the effort to prepare and execute the proof-of-concept experiments [4].

In earlier studies, we discussed different evaluation strategies that can explore the performance of unsupervised machine learning algorithms on big data platforms [35]. In this study, we introduce a test framework for big data applications. Different from the previous studies on big data testing frameworks, we introduce an abstract architecture of a testing framework that particularly focuses on testing the recommendation systems. We discuss the details of the prototype implementation of the proposed testing framework. We also show the usability of the proposed architecture by using its prototype to conduct comprehensive testing on a recommendation system designed as a big data application.

There exist studies conducted on User Interface Testing [36], [37]. Different from these studies, in this study, we only focus on designing a testing framework for recommendation systems. We observe studies that focus on designing distributed software infrastructures for varying purposes in different domains such as metadata management [38], information services [39], [40], provenance data management [41], SOA based computational science environment [42], [43]. In this study, we focus on designing distributed software architectures for testing frameworks. There exist a number of studies on recommendation systems in the literature [44]–[46]. Different from these studies, in the study, we focus on testing the recommendation systems. We observe some studies that focus on test case prioritization [47], [48]. In this study, we focus on a framework designed for testing recommendation systems.

III. Methodology

In this study, we introduce a testing framework for testing the recommendation systems using Hadoop-based big data technologies. The overview of the proposed architecture is illustrated in Figure 1.



Fig. 1. The Detailed View of the Proposed Testing Framework Software Architecture for Testing the Recommendation Algorithms.

The rectangle shape with red color in Figure 1 identifies the core software testing modules of the proposed framework. It includes various modules such as Testing Suite Module, Functional Requirements Testing Module and Non-Functional Requirements Testing Module. In order to provide modular and portable test framework software, each module is designed to be a RESTful-API based microservice.

The Testing Module is responsible for starting/stopping and monitoring the testing functions. The Functional Requirements Testing Module is responsible for testing the business functions of the big data applications. The Non-Functional Requirements Testing Module is responsible for testing the applications for latency and scalability.

In this particular study, our focus is to create a testing framework that can provide test automation functionalities for recommendation systems. As illustrated in Figure 1, we designed the testing framework to include the testing capabilities for both functional and non-functional requirements of recommendation applications. Functional Requirements Testing Methods Module includes testing functions for differing recommendation-related algorithms. These testing functions include a) test functions for item-based CF algorithm, b) test functions for user-based CF algorithm, c) test functions for ALS algorithm, d) test functions for SVD algorithm. These functions can be extended to test other recommendation-related algorithms on big datasets. Non-Functional Requirements Testing Methods Module includes testing functions with related varying non-functional metrics such as latency, scalability, and load. This module includes functions that can evaluate a number of different non-functional requirements such as system scalability under increasing data size, system scalability under increasing load, system scalability under increasing users system speed-up when increasing the number of nodes in datacenters.

Figure 1 also illustrates the application to be tested by the proposed testing framework. The rectangle shape with green color in Figure 1 identifies the application. Here, we argue that most existing recommendation applications, designed to run on large-scale data, use Hadoop-based big data processing techniques. Furthermore, these systems are implemented based on Lambda software architecture.

To this end, within the scope of this study, we design and implement an example Lambda architecture-based recommendation system, as illustrated in Figure 2. Figure 2 shows the detailed view of the module with the green colored rectangle shape in Figure 1. The Lambda architecture is a

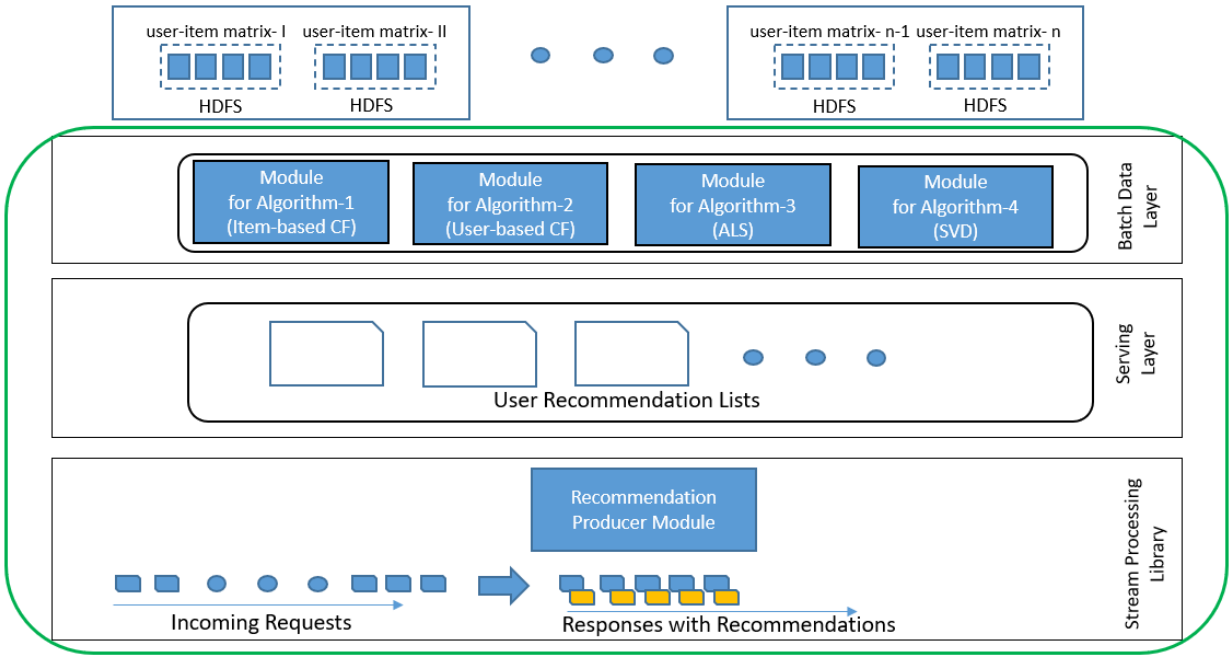


Fig. 2. An Example Illustration of Lambda Architecture-based Recommendation Application.

commonly used distributed system architecture that provides both batch processing and stream processing layers. The batch processing layer is responsible for creating the models used in the stream processing layer. The stream processing layer is responsible for generating results for each query/request based on the pregenerated models. In this study, the proposed testing framework is used to test both batch-processing and stream-processing based functionalities of this application.

The test input data for the testing framework will be stored in HDFS clusters. We should note that, since we aim to create a test framework for testing the big data applications that use Hadoop-based big data processing techniques, we assume that all the reference implementations of the proposed framework will use HDFS distributed files system as the test input data storage technology. The resulting output of the recommendation algorithms will be kept in distributed NoSQL data structures. The test results, which indicate the success/failure status of the functional requirement tests and the results of the non-functional requirement tests, will be kept as test reports in the file system. To illustrate the use of the testing framework, we provide a reference prototype implementation. We discuss the details of the prototype and how the prototype can be used to test a recommendation system in the next section in great detail.

IV. Prototype Implementation, Experiments and Evaluations

A. Prototype Implementation

In order to test the usability of the proposed testing framework, we developed a prototype implementation. The core modules of the test framework are implemented as microservices. Each microservice provides a RESTful API based programming interface. The reference prototype is implemented in Python programming language. We also implemented a graphical user interface that allows the test engineer to interact with these microservices in order to start/stop automated testing.

We used the Apache Hadoop framework for implementing the distributed data storage. For the test outputs, we utilized MongoDB framework to implement the distributed NoSQL data storage.

To implement the Lambda architecture-based recommendation application, we utilized the Apache Spark. The recommendation system is also implemented in Python programming language. Here, within the recommendation application, we implemented the following recommendation-related algorithms: user-based CF, item-based CF, ALS, and SVD. The reference prototype implementation of the testing framework is used to test the recommendation application that we developed in this study. The test reports are stored in the file system. We discuss the datasets (i.e., test inputs) and the experimental studies in the sections below.

B. Datasets

To illustrate testing of the recommendation application, as for the test input, we used real-life datasets obtained from the tourism sector. BiletBank is a flight ticket consolidator that sells the flight tickets of numerous contracted airlines to more than 3.500 member tourism agencies via its online B2B platform [49]. BiletBank provided us with a large dataset containing actual flight ticket data between 2015 and 2021. This dataset includes flight destination information for each user. We processed this dataset in order to use it in recommendation systems. Since we explicitly do not know which

flight destinations passengers like in this dataset, we extracted implicit feedback by calculating the flight frequency.

The following steps are applied to create the needed dataset:

1) We grouped the dataset by user and flight.
2) As a result of grouping, we calculated how often each user went to which destination, and obtained the rating as flight frequency.
3) Final dataset consists of tuples with the following three fields:
   a) UserID: PassengerID
   b) ItemID: DestinationID
   c) Rating: Flight Frequency

In order to test the usability of the testing framework, we obtained three datasets with different size categories: Small, Medium, and Large. Information about the details of these datasets is shown in Table I. Flight datasets naturally contain relatively large unique users (passengers) and few unique items (destination) data. Thus we needed to extend our tests with different types of datasets. To this end, we also used the MovieLens dataset [50] in our tests in addition to the flight datasets. This dataset was collected from users of the movie recommendation service MovieLens between March 1996 and September 2018 and included movie rating information (Education and Development version). Table I shows details about the used datasets.

TABLE I
PROPERTIES OF DATASETS

Dataset	# Unique Users	# Unique Items	# Tuples	Date Range	Size (MB)
Movie Lens	129,887	44,408	12,753,446	N/A	400
Flight Large	4,235,726	795	6,926,066	2015-2021	96
Flight Medium	2,708,912	673	4,313,101	2015-2019	60
Flight Small	930,320	479	1,357,230	2015-2017	19

C. Test Design

In order to implement the recommendation application, we used Apache Spark MLlib libraries. We implemented ALS, SVD, Item-based CF, and User-based CF algorithms on Spark. ALS and SVD algorithms are by default implemented on Spark MLlib libraries. For this study, we extended the Apache Spark MLlib library to include implementation for item based CF and user based CF algorithms.

During the tests, we created various Spark clusters ranging from 1-node Spark cluster to 4-node Spark cluster. Each Spark node is deployed on a virtual machine. Here, the master node was equipped with 1 CPU core and 6 GB of RAM, while the worker node was equipped with 1 CPU core and 2 GB of RAM. We ran the virtual machines on workstations equipped with an Intel Core i7-10510 processor and 16 GB of memory. In this Spark cluster environment, we tested the four recommendation system algorithms on different numbers of nodes with four different datasets. We ran each test 20 times and recorded the average runtime and standard deviation values.

The test design of the proposed testing framework includes both the testing of the functional and non-functional requirements of the recommendation systems. As for the testing of the functional requirements, we test the accuracy of recommendations produced by the recommendation application. To evaluate the accuracy, we use Root Mean Squared Error (RMSE) metric. We divide the data into two as training and test data. We produce recommendations based on the training data. Then, we analyze the difference between the recommendations obtained from the training data and real observations obtained from the test data.

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}(y_i - x_i)^2}{n}} \tag{4}$$

where y_i is the predicted value, and x_i is the actual value, i ranges from 1 to n, and n is the number of observations.

There are many destinations in our dataset that have been visited only once. This leads to the imbalanced data problem. In the data pre-processing stage of the testing, we removed the data with a very low flight frequency value from our dataset to solve this problem. We used the Flight Large dataset, whose properties are given in Table I, for accuracy tests. While conducting the accuracy tests for Item-based CF and User-based CF algorithms, we used cosine distance and Pearson correlation techniques to investigate how the similarity algorithms change the results.

D. Experimental Studies and Results

1) Scalability Tests (Non-Functional Requirements Tests) and Results: With the help of reference implementation of the proposed test framework, we conducted the scalability experiments. As a result of the scalability tests, we observed how each algorithm behaves on different numbers of nodes, and different sizes and types of datasets. Figures 3, 4, 5, and 6 show these behaviors. Tables II, III, IV, and V show the scalability results of these algorithms. We used the SpeedUp metric to analyze the scalability results. For example, below, we give the equation used to calculate the SpeedUp value obtained from experiments on Spark clusters, in the first setup running on 2-node Spark clusters and then in the next setup running on 3-node Spark clusters.

$$SpeedUp(2 \rightarrow 3) = \frac{-100 * (ExecutionTime(3) - ExecutionTime(2))}{ExecutionTime(2)} \tag{5}$$

where $ExecutionTime(x)$ refers to the time which a recommendation algorithm needs to be executed on a distributed system consisting of x number of nodes, and $SpeedUp(a \rightarrow b)$ refers to the performance gain when the number of nodes is increased from a to b. A negative SpeedUp result means loss of performance and worse scalability.

We obtain the results of the scalability experimental studies using the reference prototype implementation from the proposed testing framework. According to the results of the experimental study, we outline our findings as follows:

- As the number of nodes increases, scalability also increases, and the execution time of the recommendation algorithm decreases.
- As the size of the dataset increases, scalability can be observed more clearly. It becomes difficult to interpret scalability for small datasets. We think that this is due to the low communication cost between the nodes.
- The number of unique items and unique users in the dataset also affects the running time of each algorithm. For example, algorithms using the matrix factorization technique have nearly identical running times, even though the size of the MovieLens dataset is four times bigger than the size of the Flight Large dataset. This difference is more clearly seen in experimental studies for Item-based CF and User-based CF. These algorithms run much faster on the MovieLens dataset than on the Flight dataset.
- Algorithms generally have much higher scalability rates when setup of the number of nodes in Spark clusters changes from 1-node to 2-node, as opposed to the case when the number of nodes changes from 3-node to 4-node.

TABLE II
SPEED UP RATES (%) FOR THE ALS ALGORITHM

Dataset	**1→2**	**1→3**	**1→4**	**2→3**	**2→4**	**3→4**
MovieLens	18	25	28	7	13	5
Flight Large	18	20	21	2	3	1
Flight Medium	10	10	1	0	-12	-12
Flight Small	0	0	0	0	0	0

TABLE III
SPEED UP RATES (%) FOR THE ITEM-BASED CF ALGORITHM

Dataset	**1→2**	**1→3**	**1→4**	**2→3**	**2→4**	**3→4**
MovieLens	25	37.5	25	16	0	-20
Flight Large	16	31	32	16	20	5
Flight Medium	27	28	29	1	3	1
Flight Small	40	40	40	0	0	0

TABLE IV
SPEED UP RATES (%) FOR THE USER-BASED CF ALGORITHM

Dataset	**1→2**	**1→3**	**1→4**	**2→3**	**2→4**	**3→4**
MovieLens	25	37.5	25	16	0	-20
Flight Large	18	27	24	13	7	-4
Flight Medium	23	26	25	3	2	-1
Flight Small	20	20	0	0	-25	-25

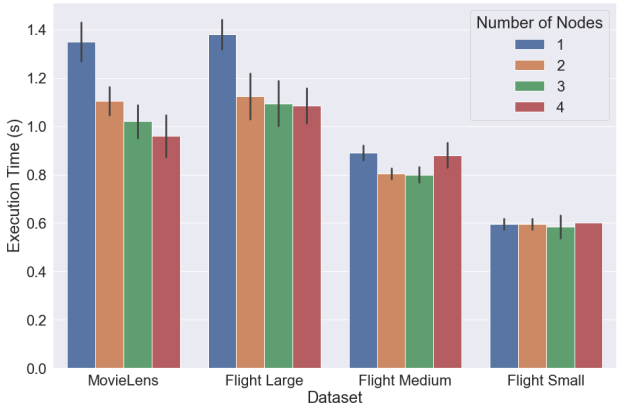


Fig. 3. Execution times for different numbers of nodes for different datasets using the ALS algorithm.

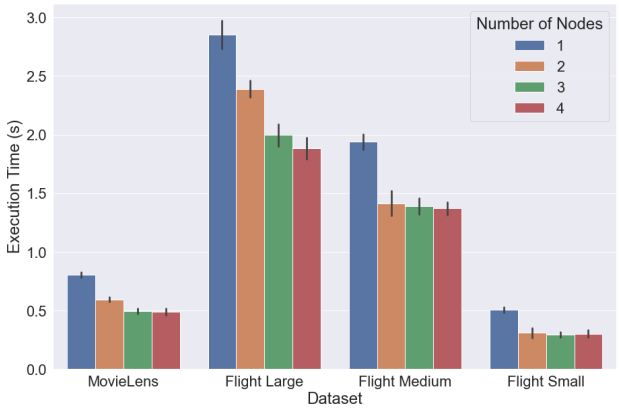


Fig. 4. Execution times for different numbers of nodes for different datasets using the Item-based CF algorithm.

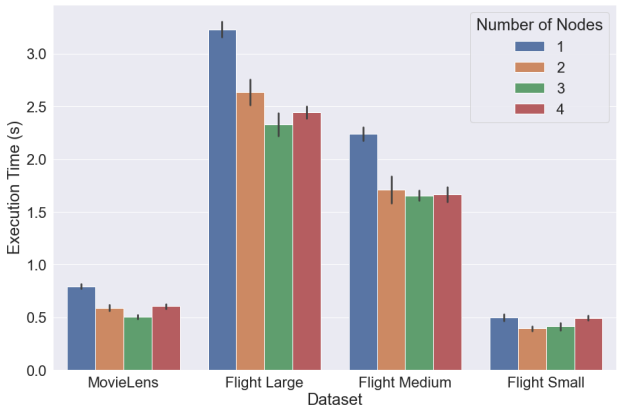


Fig. 5. Execution times for different numbers of nodes for different datasets using the User-based CF algorithm.

TABLE V
SPEED UP RATES (%) FOR THE SVD ALGORITHM

Dataset	**1→2**	**1→3**	**1→4**	**2→3**	**2→4**	**3→4**
MovieLens	26	38	43	16	23	8
Flight Large	29	39	40	13	15	2
Flight Medium	29	37	36	10	9	-1
Flight Small	29	20	25	-12	-5	6

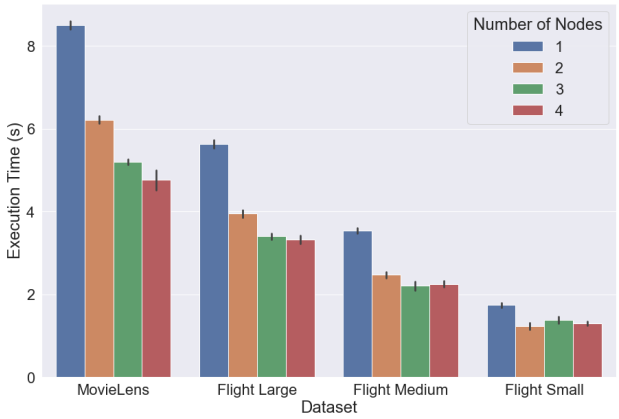


Fig. 6. Execution times for different numbers of nodes for different datasets using the SVD algorithm.

2) Accuracy Tests (Functional Requirements Tests) and Results: Producing recommendations is considered as the functional requirement of the recommendation applications. To show the use and efficiency of the Functional Requirement Testing Module of the proposed testing framework, we also tested the accuracy of the recommendation application. Here, each recommendation algorithm is applied to the Flight Large dataset. The RMSE results of each recommendation algorithm are shown in Table VI.

We obtain the results of the accuracy tests by using the reference prototype implementation from the proposed testing framework. According to the results, we outline our findings as follows: The Item-based CF with cosine distance algorithm gave the best results for the dataset. However, the results are pretty close to each other.

TABLE VI
RMSE VALUES OF THE ALGORITHMS

Algorithms	**RMSE**
Item-based CF - Cosine Distance	0.2072
Item-based CF - Pearson Correlation	0.2106
User-based CF - Pearson Correlation	0.2106
ALS	0.2115
User-based CF - Cosine Distance	0.2124
SVD	0.2179

V. CONCLUSION

The need to test big data applications in software projects to achieve high-quality software products and to meet user requirements is increasing day by day. Recommendation systems are one of the most common big data applications used both in e-science and e-commerce domains. In this study, we argue that evaluating the performance of the recommendation systems for various metrics like scalability, latency, and accuracy is an important research problem, and there is a lack of studies that address this problem.

We propose a testing framework architecture that can be used for testing big data-based recommendation system applications. To illustrate the use and efficiency of the proposed framework, we implemented a reference prototype application. Within the scope of this study, we also implemented a Lambda-architecture-based recommendation application that uses Hadoop-based distributed data storage. Here, we implemented four commonly used recommendation-related algorithms: ALS, SVD, Item-based CF, and User-based CF. In this manuscript, we discuss the details of the reference implementation of the testing framework.

We used the reference prototype of the proposed testing framework to test the recommendation application. We showed how the big data-based recommendation systems are tested for both functional and non-functional requirements. We discuss the experimental study details and the results. The results show that the proposed distributed testing framework can be utilized to conduct a comprehensive experimental study for the big data-based recommendation systems.

In future studies, we will expand the use of reference prototype application to test other recommendation system algorithms and apply our proposed testing framework to other big data systems.

ACKNOWLEDGMENT

This study was supported by BiletBank R&D Center. We would like to thank BiletBank for providing us with the necessary hardware and access to their datasets.

REFERENCES

[1] R. Casado and M. Younas, "Emerging trends and technologies in big data processing," *Concurrency and Computation: Practice and Experience*, vol. 27, no. 8, pp. 2078–2091, 2015.

[2] S. Sagiroglu and D. Sinanc, "Big data: A review," in *2013 international conference on collaboration technologies and systems (CTS)*. IEEE, 2013, pp. 42–47.

[3] B. K. Jha, G. Sivasankari, and K. Venugopal, "Product recommendation system using scalable alternating least square algorithm and collaborative filtering using apache spark in e-commerce," *Annals of the Romanian Society for Cell Biology*, pp. 2611–2622, 2021.

[4] A. Alexandrov, C. Brücke, and V. Markl, "Issues in big data testing and benchmarking," in *The Sixth International Workshop on Testing Database Systems*, 2013, pp. 1–5.

[5] N. Li, A. Escalona, Y. Guo, and J. Offutt, "A scalable big data test framework," in *2015 IEEE 8th international conference on software testing, verification and validation (ICST)*. IEEE, 2015, pp. 1–2.

[6] J. Domann, J. Meiners, L. Helmers, and A. Lommatzsch, "Real-time news recommendations using apache spark," in *Conference and Labs of the Evaluation Forum*, 09 2016.

[7] F. Ricci, L. Rokach, and B. Shapira, "Introduction to recommender systems handbook," in *Recommender systems handbook*. Springer, 2011, pp. 1–35.

[8] J. Lu, D. Wu, M. Mao, W. Wang, and G. Zhang, "Recommender system application developments: a survey," *Decision Support Systems*, vol. 74, pp. 12–32, 2015.

[9] W. Agnew, M. Fisher, I. Foster, and K. Chard, "An ensemble-based recommendation engine for scientific data transfers," in *2016 Seventh International Workshop on Data-Intensive Computing in the Clouds (DataCloud)*, 2016.

[10] J. Yao, W. Tan, S. Nepal, S. Chen, J. Zhang, D. DeRoure, and C. Goble, "Reputationnet: Reputation-based service recommendation for e-science," in *IEEE Transactions on Services Computing, Issue: 3*, vol. 8, 2016.

[11] M. A. Ghazanfar and A. Prugel-Bennett, "A scalable, accurate hybrid recommender system," in *2010 Third International Conference on Knowledge Discovery and Data Mining*, 2010, pp. 94–98.

[12] N. Mishra, S. Chaturvedi, A. Vij, and S. Tripathi, "Research problems in recommender systems," *Journal of Physics: Conference Series*, vol. 1717, p. 012002, 01 2021.

[13] M. Hahsler, "recommenderlab: A framework for developing and testing recommendation algorithms," Tech. Rep., 2015.

[14] W. X. Zhao, S. Mu, Y. Hou, Z. Lin, K. Li, Y. Chen, Y. Lu, H. Wang, C. Tian, X. Pan *et al.*, "Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms," *arXiv preprint arXiv:2011.01731*, 2020.

[15] L. Yang, E. Bagdasaryan, J. Gruenstein, C.-K. Hsieh, and D. Estrin, "Openrec: A modular framework for extensible and adaptable recommendation algorithms," in *The Eleventh ACM International Conference on Web Search and Data Mining*, 2018, pp. 664–672.

[16] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[17] S. Patil, S. Deshpande, and A. D. Potgantwar, "Product recommendation using multiple filtering mechanisms on apache spark," *International Journal of Scientific Research in Network Security and Communication*, vol. 5, no. 3, pp. 76–83, 2017.

[18] J. B. Schafer, D. Frankowski, J. Herlocker, and S. Sen, "Collaborative filtering recommender systems," in *The adaptive web*. Springer, 2007, pp. 291–324.

[19] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *The 10th International Conference on World Wide Web*, 2001, pp. 285–295.

[20] R. Anil, G. Çapan, I. Drost-Fromm, T. Dunning, E. Friedman, T. Grant, S. Quinn, P. Ranjan, S. Schelter, and Ö. Yilmazel, "Apache mahout: Machine learning on distributed dataflow systems." *J. Mach. Learn. Res.*, vol. 21, pp. 127–1, 2020.

[21] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen *et al.*, "Mllib: Machine learning in apache spark," *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1235–1241, 2016.

[22] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *2008 Eighth IEEE International Conference on Data Mining*. Ieee, 2008, pp. 263–272.

[23] Y. Zhou, D. Wilkinson, R. Schreiber, and R. Pan, "Large-scale parallel collaborative filtering for the netflix prize," in *International conference on algorithmic applications in management*. Springer, 2008, pp. 337–348.

[24] K. Baker, "Singular value decomposition tutorial," *The Ohio State University*, vol. 24, 2005.

[25] G. H. Golub and C. Reinsch, "Singular value decomposition and least squares solutions," in *Linear algebra*. Springer, 1971, pp. 134–151.

[26] V. Klema and A. Laub, "The singular value decomposition: Its computation and some applications," *IEEE Transactions on automatic control*, vol. 25, no. 2, pp. 164–176, 1980.

[27] C. O. S. Sorzano, J. Vargas, and A. P. Montano, "A survey of dimensionality reduction techniques," *arXiv preprint arXiv:1403.2877*, 2014.

[28] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Incremental singular value decomposition algorithms for highly scalable recommender systems," in *Fifth international conference on computer and information science*, vol. 1, no. 012002. Citeseer, 2002, pp. 27–8.

[29] C. Sardianos, N. Tsirakis, and I. Varlamis, *A Survey on the Scalability of Recommender Systems for Social Networks*, 06 2018, pp. 89–110.

[30] A. S. Das, M. Datar, A. Garg, and S. Rajaram, "Google news personalization: Scalable online collaborative filtering," in *The 16th International Conference on World Wide Web*, ser. WWW '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 271–280.

[31] C. Seminario and D. Wilson, "Case study evaluation of mahout as a recommender platform," vol. 910, 09 2012.

[32] S. G. Walunj and K. Sadafale, "An online recommendation system for e-commerce based on apache mahout framework," in *The 2013 Annual Conference on Computers and People Research*, ser. SIGMIS-CPR '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 153–158.

[33] C. Sardianos, I. Varlamis, and M. Eirinaki, "Scaling collaborative filtering to large–scale bipartite rating graphs using lenskit and spark," in *2017 IEEE Third International Conference on Big Data Computing Service and Applications (BigDataService)*, 2017, pp. 70–79.

[34] M. J. Awan, R. A. Khan, H. Nobanee, A. Yasin, S. M. Anwar, U. Naseem, and V. P. Singh, "A recommendation engine for predicting movie ratings using a big data approach," *Electronics*, vol. 10, no. 10, p. 1215, 2021.

[35] M. Uzun-Per, A. V. Gürel, A. B. Can, and M. S. Aktas, "An approach to recommendation systems using scalable association mining algorithms on big data processing platforms: A case study in airline industry," in *2021 International Conference on INnovations in Intelligent SysTems and Applications (INISTA)*. IEEE, 2021, pp. 1–6.

[36] Y. Uygun, R. Oguz, E. Olmezogullari, and M. Aktas, "On the large-scale graph data processing for user interface testing in big data science projects," in *2020 IEEE International Conference on Big Data (Big Data)*, 2020.

[37] M. Oz, C. Kaya, E. Olmezogullari, and M. Aktas, "On the use of generative deep learning approaches for generating hidden test scripts," *International Journal of Software Engineering and Knowledge Engineering*, vol. 31(10), p. 1447–1468, 2021.

[38] M. Aktaş, G. Fox, and M. Pierce, "Managing dynamic metadata as context," in *The 2005 Istanbul International Computational Science and Engineering Conference (ICCSE2005), Istanbul, Turkey*, 2005.

[39] M. Aktas, G. Fox, and M. Pierce, "Information services for dynamically assembled semantic grids," in *The First International Conference on Semantics, Knowledge and Grid (SKG 2005), Beijing, China*, 2005.

[40] M. Aktas, G. Aydin, G. Fox, H. Gadgil, M. Pierce, and A. Sayar, "Information services for grid/web service oriented architecture (soa) based geospatial applications," in *The First International Conference on Semantics, Knowledge and Grid (SKG 2005), Beijing, China*, 2005.

[41] Y. Tas, M. Baeth, and M. Aktas, "An approach to standalone provenance systems for big social provenance data," in *12th International Conference on Semantics, Knowledge and Grids (SKG)*, 2016.

[42] G. Aydin, M. Aktas, G. Fox, H. Gadgil, M. Pierce, and A. Sayar, "Servogrid complexity computational environments cce integrated performance analysis," in *2005 6th International Workshop on Grid Computing (GRID)*, 2005.

[43] M. Pierce, G. Fox, M. Aktas, G. Aydin, H. Gadgil, Z. Qi, and A. Sayar, "The quakesim project: Web services for managing geophysical data and applications," *Journal of Pure and Applied Geophysics*, vol. 165, pp. 635–651, 2008.

[44] Z. Duzen and M. Aktas, "An approach to hybrid personalized recommender systems," in *2016 International Symposium on INnovations in Intelligent SysTems and Applications (INISTA)*, 2016.

[45] K. Tas, E. Onder, and M. Aktas, "On the implicit feedback based data modeling approaches for recommendation systems," in *2021 International Conference on Electrical, Communication, and Computer Engineering (ICECCE 2021)*, 2021.

[46] M. Aktas, M. Pierce, G. Fox, and L. D, "A web based conversational case-based recommender system for ontology aided metadata discovery," in *The Fifth IEEE/ACM International Workshop on Grid Computing*, 2004.

[47] T. Boyar, M. Oz, and M. Aktas, "A novel approach to test case prioritization for software regression tests," in *International Conference on Computational Science and Its Applications (ICCSA)*, 2021.

[48] S. Ozturk, N. Yagci, M. Aktas, M. Ozbek, and F. Paligu, "Test case prioritization related to code quality," in *The International Conference on Software Engineering Research and Practice (SERP)*, 2013.

[49] Biletbank: Turkey's leading consolidator for flight tickets. [Online]. Available: https://www.biletbank.com

[50] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, Dec. 2015. [Online]. Available: https://doi.org/10.1145/2827872